ELVA-Q 0x53

Quantum Circuit Simulator

C++17 · Dear ImGui · ImPlot · CMake

1. Introduction

ELVA-Q 0x53 is a real-time quantum circuit simulator built entirely from mathematical first principles. Every component — from complex number arithmetic through unitary gate application to projective measurement — is derived from the postulates of quantum mechanics and implemented in C++17 without reliance on any external quantum library. This report presents the mathematical framework that was solved and encoded before the first line of GUI code was written.

The simulator operates on the state-vector model of quantum computation. An n-qubit register is represented as a unit vector in a 2^n-dimensional complex Hilbert space $\mathbb{C}^{2^n}$. Quantum gates are unitary operators acting on that space. Measurement is a projective operation governed by the Born rule. These are not abstractions layered on top of a black-box — they are the literal data structures and algorithms in the source code.

2. Mathematical Foundations

2.1 Complex Number Arithmetic

All quantum amplitudes are elements of $\mathbb{C}$. The simulator implements its own Complex<double> type, defining the full algebraic structure required.

A complex number z is represented as an ordered pair of reals:

$$z = a + bi, \quad a, b \in \mathbb{R}, \quad i^2 = -1 \tag{2.1}$$

Arithmetic operations are implemented element-wise. Multiplication uses the distributive law:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i \tag{2.2}$$

The complex conjugate and squared norm (used directly for measurement probabilities) are:

$$z^* = a - bi \tag{2.3}$$

$\lvert z\rvert^2 = a^2 + b^2 = z \cdot z^*$	**(2.4)**

For phase construction — needed in rotation and Fourier gates — the polar form is implemented via Euler's formula:

$z = r \cdot e^{\wedge}(i\theta) = r(\cos\theta + i\sin\theta), \quad r \in \mathbb{R}^+$	**(2.5)**

Pre-computed constants stored in code: 0, 1, i, and 1/√2 (= INV_SQRT2), which appears in the Hadamard gate and all superposition states.

2.2 The n-Qubit Hilbert Space

A single qubit is a unit vector in $\mathbb{C}^2$, the two-dimensional complex Hilbert space. The computational basis states are:

$\lvert 0\rangle = [1, 0]^T \qquad \lvert 1\rangle = [0, 1]^T$	**(2.6)**

A general single-qubit pure state is a normalised superposition:

$\lvert\psi\rangle = \alpha\lvert 0\rangle + \beta\lvert 1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad \lvert\alpha\rvert^2 + \lvert\beta\rvert^2 = 1$	**(2.7)**

For n qubits, the state space is the tensor product of n single-qubit spaces:

$\mathcal{H}_n = \mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2 \ \text{(n times)} \ \cong \ \mathbb{C}^{\wedge}(2^n)$	**(2.8)**

The computational basis for $\mathcal{H}$□ consists of all 2^n tensor products of single-qubit basis states, labeled by binary strings:

$\lvert x_\{n-1\}\, x_\{n-2\} \cdots x_1\, x_0\rangle, \quad x_i \in \{0, 1\}$	**(2.9)**

A general n-qubit pure state is therefore:

$\lvert\Psi\rangle = \Sigma_\{x=0\}^\{2^n-1\} \ \alpha_x \lvert x\rangle, \quad \alpha_x \in \mathbb{C}, \quad \Sigma \lvert\alpha_x\rvert^2 = 1$	**(2.10)**

In code, this is exactly the amplitudes[] vector in StateVector: a dense array of 2^n complex numbers initialised to $\lvert 0\cdots 0\rangle$ = (1, 0, …, 0).

2.3 Quantum Gates as Unitary Operators

Quantum gates are linear operators $U : \mathcal{H}\square \to \mathcal{H}\square$ satisfying the unitarity condition:

$U^\dagger U = UU^\dagger = I$	**(2.11)**

Unitarity preserves the normalisation of $|\Psi\rangle$, ensuring probabilities always sum to 1. Single-qubit gates act as 2×2 complex unitary matrices. The simulator defines the Matrix2 struct:

$U = \begin{vmatrix} u_{00} & u_{01} \end{vmatrix} \in U(2), \quad \det(U) = u_{00}u_{11} - u_{01}u_{10} \in \{e^{(i\varphi)}\}$	**(2.12)**

2.3.1 Pauli Gates

The three Pauli matrices, together with the identity, form the basis of single-qubit gate space:

$I = \begin{vmatrix} 1 & 0 \end{vmatrix} \quad X = \begin{vmatrix} 0 & 1 \end{vmatrix} \quad Y = \begin{vmatrix} 0 & -i \end{vmatrix} \quad Z = \begin{vmatrix} 1 & 0 \end{vmatrix}$	**(2.13)**
$\begin{vmatrix} 0 & 1 \end{vmatrix} \quad \begin{vmatrix} 1 & 0 \end{vmatrix} \quad \begin{vmatrix} i & 0 \end{vmatrix} \quad \begin{vmatrix} 0 & -1 \end{vmatrix}$	

X is the bit-flip (NOT) gate, Z is the phase-flip gate. They satisfy $X^2 = Y^2 = Z^2 = I$ and $XYZ = iI$, forming the Pauli group.

2.3.2 Hadamard Gate

The Hadamard gate H maps computational basis states to equal superpositions — the fundamental operation for creating quantum parallelism:

$H = 1/\sqrt{2} \cdot \begin{vmatrix} 1 & 1 \end{vmatrix}, \quad H\|0\rangle = \|+\rangle = (\|0\rangle + \|1\rangle)/\sqrt{2}$	**(2.14)**
$\begin{vmatrix} 1 & -1 \end{vmatrix} \quad H\|1\rangle = \|-\rangle = (\|0\rangle - \|1\rangle)/\sqrt{2}$	

$H^2 = I$ (self-inverse). The factor $1/\sqrt{2}$ is the INV_SQRT2 constant defined in complex.hpp.

2.3.3 Phase & Rotation Gates

Phase gates apply a relative phase. The S and T gates are special cases of the phase gate $P(\theta)$:

$P(\theta) = \begin{vmatrix} 1 & 0 \end{vmatrix}, \quad S = P(\pi/2), \quad T = P(\pi/4)$	**(2.15)**
$\begin{vmatrix} 0 & e^{(i\theta)} \end{vmatrix}$	

Rotation gates Rx, Ry, Rz parametrise the continuous SU(2) manifold — the group of 2×2 unitary matrices with determinant 1. They rotate the Bloch vector by angle θ around the respective axis:

$Rx(\theta) = \exp(-i\theta X/2) = \begin{bmatrix} \cos(\theta/2) & -i\cdot\sin(\theta/2) \\ -i\cdot\sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$	**(2.16)**

$Ry(\theta) = \exp(-i\theta Y/2) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$	**(2.17)**

$Rz(\theta) = \exp(-i\theta Z/2) = \begin{bmatrix} e^{(-i\theta/2)} & 0 \\ 0 & e^{(i\theta/2)} \end{bmatrix}$	**(2.18)**

The universal single-qubit gate U3(θ, φ, λ) spans all of SU(2):

$U3(\theta,\varphi,\lambda) = \begin{bmatrix} \cos(\theta/2) & -e^{(i\lambda)}\cdot\sin(\theta/2) \\ e^{(i\varphi)}\cdot\sin(\theta/2) & e^{(i(\varphi+\lambda))}\cdot\cos(\theta/2) \end{bmatrix}$	**(2.19)**

2.4 Gate Application Algorithm

Applying a single-qubit gate U on qubit q of an n-qubit state vector requires updating exactly half the 2^n amplitude pairs. The algorithm exploits the tensor product structure of the basis:

For each computational basis index i where bit q is 0, let j = i | (1 << q). Then:

$\begin{bmatrix} \alpha_j' \\ \alpha_i' \end{bmatrix} = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix} \begin{bmatrix} \alpha_i \\ \alpha_j \end{bmatrix}, \quad \text{for all i s.t. bit q of i} = 0$	**(2.20)**

This is the GateApply::apply1() loop. It requires $O(2^n)$ time and O(1) extra memory — optimal for dense state-vector simulation.

2.4.1 Two-Qubit Gate: CNOT

The Controlled-NOT gate acts on the 4-dimensional subspace of two qubits c (control) and t (target). In the computational basis {|00⟩ , |01⟩ , |10⟩ , |11⟩ }:

$\mathrm{CNOT} = \lceil 1\ 0\ 0\ 0 \rceil = \lvert 0\rangle\langle 0\rvert \otimes \mathrm{I} + \lvert 1\rangle\langle 1\rvert \otimes \mathrm{X}$	**(2.21)**
$\lceil 0\ 1\ 0\ 0 \rceil$	
$\lceil 0\ 0\ 0\ 1 \rceil$	
$\lfloor 0\ 0\ 1\ 0 \rfloor$	

Implementation: for all basis indices i where bit c = 1 and bit t = 0, swap amplitudes[i] ↔ amplitudes[i | (1 << t)]. No matrix is explicitly stored.

2.4.2 Three-Qubit Gate: Toffoli (CCX)

The Toffoli gate is a doubly-controlled NOT — a universal reversible classical gate. It flips target qubit t iff both controls c_0 and c_1 are $|1\rangle$:

$\mathrm{CCX}\lvert c_0, c_1, t\rangle = \lvert c_0, c_1, t \oplus (c_0 \wedge c_1)\rangle$	**(2.22)**

Implementation: swap amplitudes for indices where both control bits are set and target bit is 0.

2.5 Quantum Measurement Theory

Measurement in the computational basis is described by the Born rule. The probability of observing outcome $|x\rangle$ when measuring state $|\Psi\rangle$ is:

$\mathrm{P}(x) = \lvert\langle x\rvert\Psi\rangle\rvert^2 = \lvert\alpha_x\rvert^2$	**(2.23)**

After measuring outcome x, the state collapses (post-measurement state update):

$\lvert\Psi\rangle \rightarrow \lvert x\rangle \quad \text{(wavefunction collapse)}$	**(2.24)**

The full measurement algorithm (measure_all) samples x from the distribution {P(x)} using inverse CDF with a uniform random variate r ~ U(0,1):

$\text{Find } x^* = \min\{\, x : \Sigma_\{k \le x\}\ \mathrm{P}(k) \ge r \,\}$	**(2.25)**

2.5.1 Single-Qubit Projective Measurement

To measure qubit q without collapsing other qubits, the marginal probability of outcome $|1\rangle$ is computed first:

$P(q = 1) = \Sigma_\{i : \text{bit } q \text{ of } i = 1\} \ \lvert\alpha_i\rvert^2$	(2.26)

Outcome m ∈ {0, 1} is sampled. The state is then projected and re-normalised:

$\alpha_i' = \alpha_i / \sqrt{P(q = m)} \quad \text{if bit } q \text{ of } i = m$	(2.27)
$\alpha_i' = 0 \qquad \text{otherwise}$	

This is measure_qubit(q, rng) in statevector.hpp — a textbook partial measurement implementation.

2.6 Bloch Sphere and Pauli Expectation Values

For a single-qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, the Bloch vector $r = (\langle X\rangle, \langle Y\rangle, \langle Z\rangle) \in \mathbb{R}^3$, $|r| \leq 1$ uniquely parameterises the state:

$\langle X\rangle = \langle\psi\lvert X\rvert\psi\rangle = 2\,\mathrm{Re}(\alpha^* \beta)$	(2.28)
$\langle Y\rangle = \langle\psi\lvert Y\rvert\psi\rangle = 2\,\mathrm{Im}(\alpha^* \beta)$	(2.29)
$\langle Z\rangle = \langle\psi\lvert Z\rvert\psi\rangle = \lvert\alpha\rvert^2 - \lvert\beta\rvert^2$	(2.30)

These are computed by expect_X(), expect_Y(), expect_Z() in statevector.hpp using direct amplitude sums over the full state vector (marginalised over all other qubits). The visualised Bloch arrow has Cartesian endpoint $(\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$ on the unit sphere for pure states.

3. Quantum Algorithms — Mathematical Derivations

3.1 Bell States — Maximal Entanglement

Bell states are the four maximally entangled two-qubit states. They violate the CHSH inequality maximally (Tsirelson bound $2\sqrt{2} \approx 2.828$). Starting from $|00\rangle$:

$\lvert\Phi+\rangle = (\lvert 00\rangle + \lvert 11\rangle)/\sqrt{2} \;\leftarrow\; H\otimes I \cdot CNOT_\{0\to 1\} \cdot \lvert 00\rangle$	(3.1)
$\lvert\Phi-\rangle = (\lvert 00\rangle - \lvert 11\rangle)/\sqrt{2} \;\leftarrow\; H\otimes I \cdot CNOT \cdot X_1 \cdot \lvert 00\rangle$	(3.2)
$\lvert\Psi+\rangle = (\lvert 01\rangle + \lvert 10\rangle)/\sqrt{2} \;\leftarrow\; H\otimes I \cdot CNOT \cdot X_0 \cdot \lvert 00\rangle$	(3.3)
$\lvert\Psi-\rangle = (\lvert 01\rangle - \lvert 10\rangle)/\sqrt{2} \;\leftarrow\; H\otimes I \cdot CNOT \cdot X_0 X_1 \cdot \lvert 00\rangle$	(3.4)

Schmidt decomposition of $|\Phi+\rangle$ shows Schmidt rank 2, confirming it cannot be written as a product state — it is genuinely entangled.

3.2 GHZ States — Multipartite Entanglement

The n-qubit Greenberger–Horne–Zeilinger state is the natural generalisation of $|\Phi+\rangle$ to n parties:

$\lvert GHZ_n\rangle = (\lvert 00\cdots 0\rangle + \lvert 11\cdots 1\rangle)/\sqrt{2}$	(3.5)

Prepared by H on qubit 0 followed by a cascade of CNOT_{0→k} for k = 1, …, n−1. After preparation, partial trace over any (n−1) qubits yields a maximally mixed state $\rho = I/2$, demonstrating total loss of local information to entanglement.

3.3 Quantum Fourier Transform

The QFT is the quantum analogue of the Discrete Fourier Transform. It maps computational basis states:

$QFT\lvert j\rangle = 1/\sqrt{N} \cdot \Sigma_\{k=0\}\hat{}\{N-1\}\; e\hat{}(2\pi ijk/N)\, \lvert k\rangle, \;\; N = 2^n$	(3.6)

The efficient circuit realisation uses n Hadamard gates plus n(n−1)/2 controlled phase rotations, requiring $O(n^2)$ gates versus O(N log N) operations for the classical FFT:

$\text{QFT circuit: } H_0 \cdot R_2\hat{}\{(10)\} \cdot R_3\hat{}\{(20)\} \cdot \cdots \cdot H_1 \cdot R_2\hat{}\{(21)\} \cdot \cdots \cdot SWAP$	(3.7)

where R□^{(jl)} denotes a controlled-$P(2\pi/2^k)$ gate with qubit j as control and l as target. The final SWAP layer reverses the bit order to match the standard DFT convention.

3.4 Grover's Search Algorithm

Grover's algorithm searches an unstructured database of $N = 2^n$ items in $O(\sqrt{N})$ oracle queries, compared to $O(N)$ classically. The algorithm iterates the Grover operator G:

$G = -H^{\otimes n} \cdot Z_0 \cdot H^{\otimes n} \cdot O_x$	**(3.8)**

where O_x is the oracle (marks the target state with a phase flip) and $Z_0 = 2|0\rangle\langle 0| - I$ is the phase inversion about the zero state. Each application of G rotates the state by angle 2θ in the $\{|\text{target}\rangle, |\text{rest}\rangle\}$ plane:

$\theta = \arcsin(1/\sqrt{N}) \approx 1/\sqrt{N}$ for large N	**(3.9)**

Optimal iterations: $t^* = \lfloor \pi\sqrt{N} / 4 \rfloor$	**(3.10)**

After t^* iterations, the state has amplitude ≈ 1 on the target. For the 2-qubit case (N = 4), a single iteration suffices ($t^* = 1$), giving exact success probability 1.

3.5 Quantum Teleportation

Quantum teleportation transmits an arbitrary unknown qubit state $|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle$ using a shared Bell pair and two classical bits. The protocol exploits the identity:

$\lvert\varphi\rangle\lvert\Phi+\rangle = ½ [\lvert\Phi+\rangle(I\lvert\varphi\rangle) + \lvert\Phi-\rangle(Z\lvert\varphi\rangle) + \lvert\Psi+\rangle(X\lvert\varphi\rangle) + \lvert\Psi-\rangle(ZX\lvert\varphi\rangle)]$	**(3.11)**

After a Bell-basis measurement on qubits 0 and 1 yielding bits (b_0, b_1), qubit 2 is in state $X^{b_1} Z^{b_0} |\varphi\rangle$. Applying the correction recovers $|\varphi\rangle$ exactly, without any information about α or β having been transmitted classically.

3.6 Bernstein–Vazirani Algorithm

Given oracle $O_{\square} : |x\rangle|y\rangle \to |x\rangle|y \oplus (s \cdot x)\rangle$ for hidden bitstring $s \in \{0,1\}^n$, BV recovers s in a single query (versus n classical queries). The circuit applies $H^{\otimes n}$ to the uniform superposition, calls the oracle, then $H^{\otimes n}$ again:

$H^{\otimes n} \cdot O_s \cdot H^{\otimes n} \lvert 0^n\rangle\lvert-\rangle \to \lvert s\rangle\lvert-\rangle$	**(3.12)**

The hidden string s = 101 is recovered deterministically by measuring the first 3 qubits. The ancilla qubit (initialised as $X \cdot H|0\rangle = |-\rangle$) absorbs the phase kickback.

4. Implementation Architecture

4.1 File Structure and Responsibilities

File	Type	Mathematical Role
include/complex.hpp	Header	$\mathbb{C}$ arithmetic: Eq. 2.1–2.5, polar form, conjugate, norm²
include/statevector.hpp	Header	$\mathcal{H}$ representation: Eq. 2.10, Born rule: Eq. 2.23–2.27, expectations: Eq. 2.28–2.30
include/gates.hpp	Header	U(2) matrices: Eq. 2.12–2.19, GateApply: Eq. 2.20–2.22
include/circuit.hpp	Header	Gate sequence composition, run() executor, RunResult
include/algorithms.hpp	Header	Circuit factories for Sec. 3.1–3.6 algorithms
include/app_state.hpp	Header	Shot engine (repeated Eq. 2.25), AppState global
src/main.cpp	Source	GLFW/ImGui/OpenGL init, render loop
src/app.cpp	Source	SetupStyle(), RenderFrame(), panel layout
src/panels/state_panel.cpp	Source	State vector display, ImPlot probability histogram
src/panels/bloch_panel.cpp	Source	Bloch sphere: Eq. 2.28–2.30 visualised in $\mathbb{R}^3$
src/panels/algo_panel.cpp	Source	One-click algorithm loader

4.2 State Vector Complexity Analysis

The computational complexity of the simulation scales with the Hilbert space dimension 2^n:

Memory: $M(n) = 2^n \cdot 2 \cdot 8$ bytes $= 2\text{^}(n+4)$ bytes (two doubles per amplitude)	(4.1)
Single-gate op: $T_1(n) = O(2^n)$ amplitude pairs updated	(4.2)
Full circuit: $T(n, d) = O(d \cdot 2^n)$ for circuit depth d	(4.3)
Shot simulation: $T_shots(n, d, s) = O(s \cdot d \cdot 2^n)$ for s shots	(4.4)

At the maximum of n = 20 qubits: 2^{20} = 1,048,576 amplitudes requiring 16 MB of memory — well within the limits of a modern desktop. For n = 30, this would reach 16 GB, marking the practical boundary of dense state-vector simulation.

5. Graphical User Interface

The GUI is built with Dear ImGui v1.91.6 and ImPlot v0.16 over a GLFW 3.4 + OpenGL 3.0 backend. Six dockable panels expose the mathematical engine to the user in real time. The layout is locked (io.IniFilename = nullptr) and rendered at 60 fps via a standard render loop in main.cpp.

5.1 Gate Palette

Buttons for all gate families: Pauli (I, X, Y, Z), Clifford (H, S, T, SX), rotation (Rx, Ry, Rz with angle sliders), two-qubit (CNOT, CZ, SWAP), and three-qubit (Toffoli). Each button appends a GateOp to Circuit::ops.

5.2 Circuit Panel

Renders the circuit as a qubit × time-step table. Run Circuit executes Eq. 2.20–2.22 in sequence. Run Shots executes the circuit s times (Eq. 2.25) and collects a histogram of measurement outcomes.

5.3 State Panel

Displays $|\alpha_x|^2$, $\mathrm{Re}(\alpha_x)$, $\mathrm{Im}(\alpha_x)$ for every basis state $|x\rangle$, a bar chart of probabilities via ImPlot, and per-qubit marginals $P(q_i = 1)$ from Eq. 2.26.

5.4 Bloch Sphere Panel

Computes the Bloch vector (Eq. 2.28–2.30) for the selected qubit and renders it as a 3D arrow on a wireframe sphere using ImPlot's 3D scatter. The Bloch vector has unit length for pure states and shrinks toward the origin for mixed states (post-measurement states are always pure here).

5.5 Algorithm Panel

One-click loaders for all algorithms in Section 3. Selecting an algorithm calls load_algorithm(), which resets the qubit count, loads the pre-built Circuit, and runs it — demonstrating the mathematical results immediately.

5.6 Log Panel

Scrolling status log with colour-coded entries (cyan for system, green for execution, amber for algorithm loads). Retains the last 100 messages.

6. Design Decisions and Observations

6.1 Strengths

- Complete mathematical self-containment: every quantum operation is derived from postulates and implemented in plain C++17 with no quantum SDK dependency.
- Dense state-vector formalism is exact for pure states and handles all entanglement structures correctly, including GHZ and Bell states.
- Bloch sphere expectation values use the full multi-qubit state vector (marginalising correctly via Eq. 2.28–2.30), not a reduced density matrix approximation.
- The fluent circuit builder API (circuit.h(0).cnot(0,1)…) closely mirrors Dirac notation for circuit composition.
- Multi-shot engine correctly reinitialises $|\Psi_0\rangle$ each shot, avoiding bias from residual post-measurement state.

6.2 Limitations and Possible Improvements

- Dense state-vector is limited to n ≤ 20 qubits. Matrix Product State (MPS) or stabiliser formalism would extend range to 50–1000+ qubits for restricted circuit classes.
- No density matrix support: mixed states, decoherence, and noise channels (Kraus operators) are not modelled.
- Font paths are Windows-only (C:\Windows\Fonts\consola.ttf); a cross-platform font loader would improve portability.
- QFT circuit uses a simplified CU parameterisation; a production implementation should use the explicit controlled-P(2π/2^k) gates.
- No unit test suite — adding catch2 or GoogleTest with reference state-vector checks would validate correctness of gate matrices and measurement statistics.

7. Summary

ELVA-Q 0x53 demonstrates that the postulates of quantum mechanics — complex Hilbert spaces, unitary evolution, Born rule measurement, and tensor product composition — can be translated directly into a working desktop simulator with no mathematical abstraction layer omitted. The equations in Section 2 are not approximations or analogies; they are the exact operations performed on the amplitude vector at runtime.

The simulator correctly implements all standard single- and multi-qubit gates, projective measurement with state collapse, multi-shot statistics, Bloch sphere expectation values, and a library of canonical quantum algorithms. It serves as both a pedagogical tool — showing the connection between mathematical formalism and executable code — and a capable demonstration platform for quantum circuit behaviour.